第12堂｜進階實務：Vibe Coding 與 Agent 系統協作

學習目標

完成本堂課後，學員將能夠：

- 設計並實作**多 Agent 協作系統**，讓多個 Agent 分工完成複雜任務
- 建立一個 AI 全球新聞情報收集系統，整合跨語系資料
- 使用 AI 對異質資料進行趨勢分析
- 將分析結果以視覺化圖表方式呈現

核心概念說明

1. 多 Agent 協作系統

單一 Agent vs. 多 Agent 系統

	單一 Agent	多 Agent 系統
適合場景	單一明確的線性任務	複雜、多面向的任務
優點	設計簡單、好維護	可平行處理、分工明確
缺點	複雜任務時上下文過長	協調邏輯較複雜

多 Agent 協作的典型架構：

```
┌──────────────────────────────────────┐
│          協調者 Agent                 │
│      (Orchestrator Agent)            │
│  接收任務，拆解並分配給子 Agent        │
└──────────────────────────────────────┘
          │            │
          ▼            ▼
┌──────────────┐  ┌──────────────┐
│ 資料搜集 Agent │  │ 資料分析 Agent │
│              │  │              │
│ 搜集各地區新聞 │  │ 分析趨勢與模式 │
└──────────────┘  └──────────────┘
          │            │
          └────────────┘
                ▼
          ┌──────────────┐
          │ 報告生成 Agent │
          │ 整合結果，輸出 │
          └──────────────┘
```

---

2. 全球新聞情報系統的需求分析

系統目標：

每天自動收集全球主要媒體的新聞，依主題分類後，提供趨勢分析和異常訊號偵測。

系統需要解決的挑戰：

挑戰	說明	解決方案
多語言	英文、中文、日文等不同語言	使用多語言 LLM 或翻譯 API
資料量大	每天可能有數百篇新聞	分批處理 + 重要性過濾
去重複	同一事件被多家媒體報導	語意相似度過濾
趨勢識別	找出突然增加的話題	關鍵字頻率時序分析
來源可信度	不同媒體的可信程度不同	建立媒體可信度白名單

---

3. 跨語系資料整合

多語言處理的兩種策略：

策略 A：翻譯後統一處理

```
外語文章 → 翻譯 API（Google Translate / DeepL）→ 翻譯成中文 → 統一分析
優點：分析邏輯簡單
缺點：翻譯費用、可能損失一些細節
```

策略 B：使用多語言 LLM 直接處理

```
外語文章 → 直接送給多語言 LLM 分析 → 要求 LLM 以中文輸出結果
優點：保留語意精確度、省去翻譯步驟
缺點：需要 LLM 支援多語言能力（GPT-4o、Claude 均支援）
```

本課程採用策略 B（更簡單且效果好）：

```
Prompt 範例：
「以下是一篇英文/日文/韓文新聞，請你：
1. 判斷這篇新聞的主題類別
2. 提取關鍵資訊（誰、做了什麼、影響是什麼）
3. 給出一個 100 字以內的繁體中文摘要
4. 評估這則新聞的重要性（1-5分）和可信度（高/中/低）
請以 JSON 格式回傳。」
```

---

4. 趨勢分析的基本方法

什麼是「趨勢」？

趨勢分析的目的是回答：「關於某個主題，最近的討論是增加還是減少？有沒有突然的爆發？」

簡單的趨勢分析方法：

方法	說明	適用情境
關鍵字頻率統計	計算特定關鍵字在不同時間段出現的次數	追蹤熱門話題的消長
新聞數量時序圖	以折線圖呈現每日/每週的報導數量	識別討論高峰
情感趨勢	追蹤特定主題的情感從正面變負面或反之	品牌聲譽監控
異常偵測	找出短時間內大幅增加的主題	快速識別突發事件

趨勢視覺化的常用圖表：

圖表類型	適合展示
折線圖	隨時間的數量變化
熱力圖	主題 × 時間的強度分佈
詞雲	高頻關鍵字的視覺化
長條圖	各主題的報導量比較

---

5. 課程進階技術整合

本堂課是整個課程前半段的綜合應用，將整合以下技術：

```
技術來源            整合應用
──────────────────────────────────────────
第04堂（Web爬蟲）   ──→  自動抓取各媒體網站新聞
第07堂（API整合）   ──→  NewsAPI + 翻譯 API 資料取得
第03堂（資料處理）  ──→  多語言新聞彙整與去重
第09堂（情報分析）  ──→  AI 主題分類與重要性評估
第10堂（視覺化）    ──→  趨勢圖表生成
```

實作步驟（逐步教學）

實作一：設計多 Agent 協作架構

步驟 1：向 GitHub Copilot 描述協調者 Agent

```
請幫我設計一個多 Agent 協作的新聞情報系統，包含以下三個 Agent：

Agent 1 - 資料搜集 Agent：
- 從 NewsAPI 搜集英文、德文、日文新聞
- 每個語言最多取 30 篇
- 回傳標準化的新聞清單 (JSON)
```

```
Agent 2 - 分析 Agent：
- 輸入：新聞清單
- 對每篇新聞：分類主題、提取重點、輸出中文摘要、評估重要性
- 去除重複報導（同一事件的不同媒體報導）
- 回傳分析後的結構化清單

Agent 3 - 報告 Agent：
- 輸入：分析後的清單
- 生成主題分佈統計
- 生成今日重要新聞摘要（Top 5）
- 輸出 Excel 報告和趨勢圖表

協調者邏輯：
- 依序呼叫三個 Agent
- 處理每個 Agent 可能的失敗
- 紀錄整個流程的執行時間
```

實作二：建立趨勢分析模組

步驟 1：設計趨勢分析函式

```
請幫我建立一個新聞趨勢分析函式：
1. 輸入：過去 7 天的新聞分析結果（每天一個 JSON 檔案）
2. 計算每個主題類別每天的報導數量
3. 識別「異常增加」的主題（今日數量 > 過去7日平均值的 2 倍）
4. 生成以下視覺化圖表（儲存為 PNG）：
   a. 折線圖：各主題類別過去7天的報導量趨勢
   b. 長條圖：今日各主題的報導數量比較
   c. 詞雲：今日所有新聞標題的高頻關鍵字
5. 輸出一份「趨勢摘要」JSON，包含：
   - 今日最熱門主題 Top 3
   - 異常增加的主題清單
   - 與昨日相比的整體變化
```

步驟 2：安裝視覺化套件

```
pip install matplotlib
pip install wordcloud
pip install jieba   # 中文分詞（用來正確切分中文詞彙）
```

實作三：建立排程執行機制

步驟 1：設定定時執行

```
請幫我建立一個排程腳本：
使用 Python 的 schedule 套件，讓系統每天早上 7:00 自動執行：
1. 搜集昨天的新聞資料
2. 執行分析
3. 生成報告和圖表
4. 透過 LINE Notify 發送今日重要新聞摘要（Top 3）
5. 記錄執行結果到 log 檔案

請也加入以下功能：
- 如果某天執行失敗，發送失敗通知
- 保留最多 30 天的歷史資料
- 超過 30 天的舊資料自動打包壓縮後封存
```

程式碼範例（虛擬碼）

協調者 Agent

```
函式 新聞情報系統主流程():
    開始時間 = 記錄現在時間()
    顯示 "=== 開始執行全球新聞情報系統 ==="

    # Agent 1：搜集資料
    顯示 "Step 1/3: 搜集新聞資料..."
    嘗試:
        原始新聞 = 資料搜集Agent(
            語言清單=["en", "de", "ja"],
            每語言篇數=30,
            關鍵字清單=["AI", "semiconductor", "trade", "energy"]
        )
        顯示 f"搜集完成：共 {len(原始新聞)} 篇"
    如果發生錯誤:
        記錄錯誤("資料搜集 Agent 失敗")
        發送失敗通知("資料搜集失敗，請手動確認")
        結束

    # Agent 2：分析資料
    顯示 "Step 2/3: 分析新聞內容..."
    嘗試:
        分析結果 = 分析Agent(原始新聞)
        顯示 f"分析完成：{len(分析結果)} 篇新聞（已去重複）"
    如果發生錯誤:
        記錄錯誤("分析 Agent 失敗")
        # 分析失敗時，儲存原始資料供後續手動處理
        儲存(原始新聞, "待分析原始資料.json")
        結束

    # Agent 3：生成報告
    顯示 "Step 3/3: 生成報告..."
    報告路徑 = 報告Agent(分析結果)
```

```
    # 執行完成
    執行時間 = 計算執行時間(開始時間)
    顯示 f"=== 完成！耗時 {執行時間} 秒 ==="
    顯示 f"報告路徑：{報告路徑}"

    # 發送每日摘要
    摘要 = 生成每日摘要(分析結果)
    發送LINE通知(摘要)
```

趨勢分析與視覺化

```
載入 pandas, matplotlib, collections 套件

函式 生成趨勢圖(7天新聞資料):
    # 整理每天每主題的數量
    趨勢資料 = {}
    對 每天 在 7天新聞資料 中:
        日期 = 每天["日期"]
        對 每篇新聞 在 每天["新聞"] 中:
            主題 = 每篇新聞["主題"]
            如果 主題 不在 趨勢資料 中:
                趨勢資料[主題] = {}
            趨勢資料[主題][日期] = 趨勢資料[主題].get(日期, 0) + 1

    # 繪製折線圖
    圖, 軸 = 建立圖表(大小=(14, 7))
    對 每個主題, 資料 在 趨勢資料.items():
        日期清單 = 排序(資料.keys())
        數量清單 = [資料[日期] for 日期 in 日期清單]
        軸.畫折線(日期清單, 數量清單, 標籤=主題, 線條粗細=2)

    軸.設定標題("過去7天主題報導量趨勢")
    軸.設定X軸標題("日期")
    軸.設定Y軸標題("報導篇數")
    軸.顯示圖例()
    軸.旋轉X軸標籤(45度)
    儲存圖表("趨勢折線圖.png", dpi=150, 緊密排版=True)

    # 識別異常增加的主題
    異常主題 = []
    對 每個主題 在 趨勢資料 中:
        今日數量 = 今天的數量(趨勢資料[主題])
        過去6日平均 = 計算平均(過去6天的數量)
        如果 今日數量 > 過去6日平均 * 2:
            異常主題.加入({
                "主題": 主題,
                "今日": 今日數量,
                "平均": 過去6日平均,
                "增幅": f"{(今日數量/過去6日平均 - 1)*100:.0f}%"
            })
```

```
回傳 異常主題
```

練習題 / 作業

課堂練習

練習 1：Agent 分工設計

針對以下複雜任務，設計一個多 Agent 系統（描述每個 Agent 的職責和輸入/輸出）：

任務：「每週自動生成一份競品分析報告，包含主要競爭對手的最新動態、定價變化、和市場輿論」

Agent 名稱	職責	輸入	輸出

練習 2：趨勢圖解讀

假設你看到以下趨勢：「AI 晶片」這個主題在過去 7 天的報導量：

週一：5篇、週二：4篇、週三：6篇、週四：5篇、週五：4篇、週六：5篇、**週日：23篇**

- 這意味著什麼？
- 作為企業決策者，你的下一步行動是什麼？
- 如何設計 Alert 機制讓系統自動通知你？

課後作業

1. **執行完整系統**：在自己的環境中執行課堂建立的新聞情報系統，搜集並分析一天的新聞，觀察輸出的報告品質，記錄至少 3 個可以改進的地方。

2. **趨勢分析設計**：設計一個「競品監控」版本的趨勢分析，指定你的主要競爭對手（可以是任何行業），說明要追蹤哪些關鍵字、多久執行一次、異常觸發的標準是什麼。

延伸閱讀

多 Agent 框架：

- LangGraph — LangChain 的多 Agent 協作框架 — https://langchain-ai.github.io/langgraph/
- AutoGen（Microsoft）— 多 Agent 對話框架 — https://microsoft.github.io/autogen/

資料視覺化：

- matplotlib 官方文件 — https://matplotlib.org/stable/gallery/（有大量範例）
- seaborn — 更美觀的統計圖表 — https://seaborn.pydata.org/
- wordcloud 套件 — https://github.com/amueller/word_cloud

排程執行：

- schedule 套件 — https://schedule.readthedocs.io/
- APScheduler — 更強大的排程工具 — https://apscheduler.readthedocs.io/